

Asteroids no Scratch

Um remake do clássico da Atari de 1979, com um easter egg escondido, e o que construir ele me ensinou sobre lógica de programação.

Daniel Godri Neto

27 de setembro de 2026

1 · Contexto

Por que o Asteroids

2 · Reflexão

Playtest, simplicidade e o que levo

3 · O jogo

Anexo A: telas, regras e o easter egg

4 · Lógica e bugs

Anexos B e C: conceitos da aula e soluções

Por que o Asteroids

Escolhi o Asteroids porque gosto muito de jogar e porque é um jogo que jogo desde muito novo. Foi meu pai quem me ensinou, numa máquina de fliperama, quando eu tinha uns 6 anos, por volta de 2012 e 2013.

Por isso o jogo segue as regras daquele fliperama: as pedras que se dividem, o disco voador que mira em você, o hiperespaço que às vezes falha e a vida extra a cada 10.000 pontos. Onde uma regra do original deixava o jogo confuso, preferi simplificar (veja a Reflexão). O projeto levou cerca de 2 a 3 semanas.

Não é o meu primeiro jogo. Na PUC-PR, em equipe, fiz o Jokenpô, que ficou em 1º lugar, e a Batalha Naval, que ficou em 2º, na disciplina de Raciocínio Algorítmico. Em Experiência Criativa, fiz com mais dois colegas o *Ecos da Vanguarda*, um jogo top-down no Construct 3, com Game Design Document e publicado no itch.io. Também já prototipei um party game, o *Don't Blow Ur Head Off*, em Scratch, Construct 3, GameMaker e Godot, e crio mods para jogos. É daí que vem a minha experiência com game design.

Recorte do projeto: um Asteroids fiel ao fliperama onde importa e mais simples onde o original confundia, feito com os blocos do Scratch, com um segundo estilo visual escondido nas configurações.

REFLEXÃO

O que construir um jogo no Scratch me ensinou

O que mais me orgulha no projeto é o estilo Nyan Cat, o meu easter egg. Achei ele único, e desenhar a pixel art foi ótimo. Gosto de esconder easter eggs principalmente por causa do filme *Jogador Nº 1*: no final, a história é a busca por um easter egg, e o filme mostra como se chega ao primeiro easter egg criado, escondido no *Adventure*, do Atari.

As partes mais difíceis foram a nave nascendo dentro de uma pedra e os clones. Clones parecem simples, mas recebem as mesmas mensagens que o original, levam os sons junto quando são apagados e disputam as mesmas variáveis. Cada um desses problemas pediu uma solução diferente (Anexo C).

Quase desisti

Teve um momento em que eu tive um problema tanto na colisão quanto no movimento: não estava ficando parecido com o original. Pensei em desistir do projeto e fazer outro; eu estava quase desesperado. Só que decidi tentar mais uma vez e acabei conseguindo fazer um progresso bem bom. Por causa disso eu quis continuar o projeto. Se não fosse por essa correção, eu teria desistido.

Isso mostra que, mesmo depois de já ter feito vários jogos, vários GDDs e vários planejamentos, às vezes você vai ter momentos de dúvida e vai querer desistir de um projeto. Mas é importante continuar. Às vezes o jogo não vai ficar o melhor, mas a gente precisa tentar mais uma vez.

O playtest com meu irmão

Diferente do *Ecos da Vanguarda* e dos jogos de Raciocínio Algorítmico, que fiz em equipe, este projeto eu fiz sozinho. Por isso o playtest foi a forma de ter outro olhar sobre o jogo. Meu irmão, de 14 anos, jogou, chegou bem longe e se divertiu muito. Ele achou o jogo muito bom para um jogo de Scratch e encontrou um bug que eu não tinha visto: clicando muitas vezes seguidas no botão de estilo, o jogo começava a travar visualmente. Com o bug encontrado, consegui corrigi-lo.

Quando ele achou o bug, eu me senti muito bem, porque isso significava que o jogo tinha ficado melhor por causa do playtest e do feedback dele. Li dois livros que falam disso: *Half-Life 2: Raising the Bar*, de David Hodgson, que mostra como a Valve faz playtests o tempo todo, e é por isso que os jogos dela parecem tão polidos; e *A Arte do Game Design*, de Jesse Schell, que fala da importância do playtesting e de como fazer do jeito certo. Apliquei essas dicas nesse teste. É a mesma lição da formação de design: testar com outra pessoa mostra o que quem fez não enxerga. E acho que playtesting é um assunto difícil de entender se você não vai lá e faz, e sempre dá para ficar melhor nisso.

Polimento

Depois que o jogo já funcionava, acrescentei pequenos retornos visuais para cada ação: a pontuação aparece subindo no lugar onde a pedra ou o disco foi atingido, cada onda começa com um aviso "WAVE N" no centro da tela, a tela pisca de leve quando a nave explode, e os asteroides menores nascem um pouco maiores e "encolhem" para o tamanho certo.

Simplificar

Também aprendi sobre game design, sobre o que faz um jogo ser bom. Os sons eram exagerados e algumas funções eram complexas demais: às vezes, a função mais simples é a melhor. Em vez de explicar o jogo, o próprio jogo faz isso por você, de forma natural. Por isso, em dois pontos, mudei o fliperama original:

Decisão	No fliperama original	No meu jogo, e por quê
Tiros sem dar a volta na tela	Os tiros da nave saem por uma borda e voltam pela outra.	Os tiros somem ao chegar na borda. Tiros voltando pelo outro lado deixavam o jogo um pouco confuso.
Sem som de batimento e de disco	Um "batimento" de fundo acelera durante a onda, e o disco voador tem um som próprio enquanto está na tela.	Tirei os dois. Os sons ficavam exagerados e, pela minha experiência com game design, não faziam tanta diferença.

Às vezes a simplicidade é uma bênção.

Próximos passos

Com mais tempo, eu adicionaria mais estilos e talvez um modo de jogo diferente. O mais ambicioso seria um pequeno roguelike: você escolheria entre itens e habilidades diferentes e iria passando por fases cada vez mais difíceis, com chefes.

Já comecei pelo primeiro passo: o modo Build, inspirado em *Vampire Survivors* e *Mycopunk*. Depois de cada onda, o jogador escolhe 1 de 3 upgrades sorteados, que se acumulam em níveis e formam uma build diferente a cada partida (Anexo A). O que falta para o roguelike completo são as fases diferentes e os chefes.

O que levo

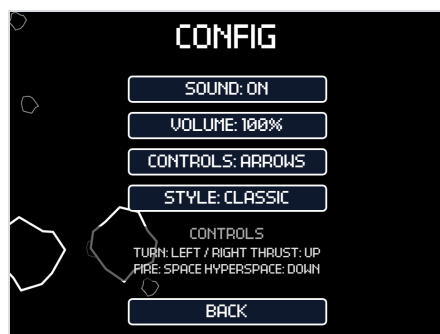
O Scratch é uma linguagem muito simples, então é preciso ser muito criativo e aberto a soluções diferentes. Quando não existe um bloco pronto para o que você quer, você precisa montar a solução com o que tem. Isso me ensinou muito sobre lidar com problemas usando abordagens diferentes. Um exemplo foi a nave nascendo dentro da pedra: o Scratch não tem um bloco que diga se o centro da tela está livre, então usei um círculo invisível que verifica se está tocando em alguma coisa.

As telas do jogo

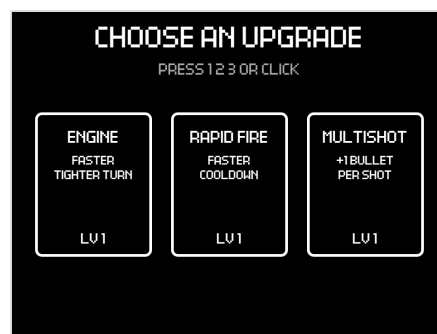
Os anexos mostram o jogo e os detalhes técnicos citados na reflexão. Abaixo, capturas do projeto rodando: o jogo tem dois modos (Arcade e Build) e dois estilos visuais, que mudam toda a arte e todos os sons.



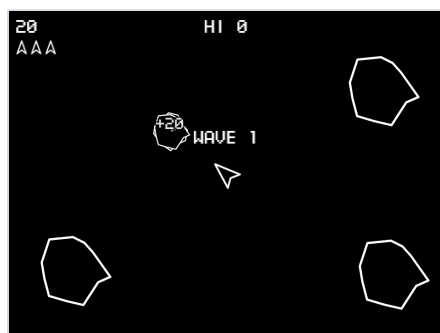
1. Menu. PLAY, BUILD MODE, CONFIG e CREDITS, com asteroides passando ao fundo e o recorde no topo.



2. Configurações. Som, volume em 4 níveis, setas ou WASD e o estilo visual. A ajuda de controles muda junto.



3. Modo Build. Depois de cada onda, o jogador escolhe 1 de 3 upgrades, que se acumulam e formam uma build.



4. Estilo clássico. O aviso "WAVE 1" e o "+20" subindo no lugar onde a pedra foi atingida e se dividiu.



5. Estilo nyan. A nave vira o Nyan Cat com rastro de arco-íris, e os asteroides viram novelos de lã.



6. Fim de jogo. No estilo nyan, o "GAME OVER" ganha um "meow...".

Regras e controles

Como jogar

Ação	Setas (padrão)	WASD
Girar a nave	← e →	A e D
Acelerar	↑	W
Atirar	Espaço	Espaço
Hiperespaço	↓	S

A nave tem inércia: ela continua deslizando depois de acelerar e perde velocidade aos poucos. Tudo que sai por uma borda da tela volta pela borda oposta, menos os tiros do jogador, que somem ao chegar na borda.

Regras do fliperama

- **Ondas:** 4 asteroides na primeira, depois 6, 8, 10 e no máximo 11, como no original.
- **Asteroides que se dividem:** o grande vira dois médios, o médio vira dois pequenos, e o pequeno é destruído. Valem 20, 50 e 100 pontos.
- **Discos voadores:** o grande (200 pontos) atira para qualquer lado; o pequeno (1.000 pontos) mira na nave.
- **Tiros:** um tiro por toque na tecla, no máximo 4 na tela ao mesmo tempo.
- **Hiperespaço:** teletransporta a nave para um lugar aleatório, mas em 1 de cada 8 vezes ele falha e a nave explode.
- **Vidas:** 3 no começo e uma vida extra a cada 10.000 pontos. Ao perder uma vida, a nave só volta quando o centro da tela está livre.
- **Recorde:** o jogo guarda a maior pontuação e pede suas iniciais, como nos fliperamas.

Modo Build

Um segundo modo, no menu. Depois de cada onda, o jogo pausa e mostra 3 upgrades sorteados; o jogador escolhe um clicando ou apertando 1, 2 ou 3. Os upgrades se acumulam em níveis: tiro múltiplo (até 3 tiros), tiro rápido (segurar para atirar), tiro perfurante (atravessa pedras), motor (mais empuxo e giro), escudo (absorve batidas e recarrega a cada onda), hiperespaço seguro (nunca falha) e vida extra. O modo Arcade continua igual ao fliperama.

O easter egg: em Configurações, o estilo troca de "classic" para "nyan". A nave vira o Nyan Cat, os asteroides viram novelos de lã e o disco voador vira um cachorro num disco. Todos os sons também mudam. Desenhei os novelos e o cachorro no Aseprite.

Os conceitos da aula dentro do jogo

Conceito	Onde aparece no jogo
Variáveis	score, lives, level e rocks controlam a partida. As configurações (sound, volume, controls, style) ficam em variáveis que a bandeira verde não zera, e por isso duram entre partidas.
Condicionais	Cada asteroide pergunta "estou tocando num tiro?", "na nave?", "no disco?". O tamanho decide a pontuação (se tier = 3, soma 20; se = 2, soma 50...). No fim da partida: se score > high score, o jogo pede as iniciais.
Laços de repetição	<i>sempre</i> : o laço principal da nave (girar, acelerar, atirar, checar colisão). <i>repita até</i> : o tiro anda até life chegar a 0. <i>espere até</i> : a próxima onda só começa quando rocks = 0.
Eventos e mensagens	O jogo é uma sequência de estados trocados por mensagens: menu → start game → new wave → game over → menu. Cada sprite só reage às mensagens que lhe interessam.
Clones	Asteroides, tiros, partículas de explosão, números de pontuação, rastro de arco-íris e botões são clones. Um asteroide atingido cria dois clones menores de si mesmo e depois se apaga.
Listas	As listas fx e pops funcionam como filas: cada explosão entra com x, y e tipo, e um único sprite vai tirando da fila e criando as partículas ou o número de pontos. No modo Build, a lista pool guarda os upgrades disponíveis, e o jogo sorteia 3 diferentes dela.
Operadores	Para a nave andar para onde está apontando, uso seno e cosseno da direção dela. Para ela ir parando aos poucos, a velocidade é multiplicada por 0,98 a cada quadro.
Aleatoriedade	Posição e direção dos asteroides, a falha do hiperespaço (sorteio de 1 a 8) e o momento em que o disco aparece.
Blocos personalizados	A lógica fica dividida em blocos com nome, como steer, shoot, hyperspace, die, split e wrap. O placar é desenhado com a caneta por um bloco que roda "sem atualizar a tela", para não piscar.

Exemplo: como um asteroide se divide

```

defina split
  se award = 1 então some a pontuação do tamanho atual
  se tier > 1 então
    mude tier por -1 (fica um tamanho menor)
    mude speed para speed × (1,2 a 1,6)
    mude safe para 5 (proteção: ignora o tiro que o dividiu)
    gire e crie clone de mim mesmo
    gire para o outro lado e crie clone de mim mesmo
  senão
    mude rocks por -1 (era o menor: só desaparece)
  apague este clone

```

É um exemplo pequeno, mas junta quase tudo da aula: variável, condicional, operador, aleatoriedade e clone. A variável safe surgiu de um bug: sem ela, os dois filhotes nasciam ainda encostados no tiro e se dividiam de novo no mesmo instante.

Bugs e como resolvi

Os principais bugs que encontrei enquanto fazia o jogo. O bug encontrado no playtest com meu irmão está na Reflexão.

Problema	O que acontecia	Como resolvi
Nave nascendo dentro de uma pedra	Depois de perder uma vida, a nave voltava no centro mesmo com um asteroide passando por ali, e explodia na hora.	Um sprite invisível em forma de círculo (SafeZone) vai para o centro, e a nave só volta quando ele não está tocando em nenhuma pedra, disco ou tiro, como no fliperama.
Clones recebendo mensagens	Quando uma mensagem como menu era enviada, todos os clones também a recebiam e executavam o mesmo código do original, criando asteroides e botões repetidos.	Cada clone guarda <code>clone = 1</code> . Os scripts de troca de tela começam com "se clone = 1, apague este clone", e só o original monta a tela nova.
Clones cortando o som	O som de explosão tocava no clone do asteroide, e quando o clone era apagado, o som parava no meio.	Todos os sons passaram a tocar de um único sprite, Sfx, que nunca é apagado. Os outros só mandam a mensagem, como <code>sfx boom 1</code> .
Clones escrevendo ao mesmo tempo	Várias pedras explodindo no mesmo quadro escreviam a posição na mesma variável, e as partículas apareciam no lugar errado.	Troquei a variável pela lista <code>fx</code> , uma fila: cada explosão entra na fila e é atendida na ordem.

Créditos

- **Asteroids**, Atari, 1979: jogo original em que as regras foram baseadas.
- **Nyan Cat**, criado por Chris Torres (2011): o estilo "nyan" é uma homenagem de fã.
- **Jogador Nº 1** (*Ready Player One*), dirigido por Steven Spielberg (2018): a inspiração para o easter egg.
- **Scratch** (scratch.mit.edu), MIT Media Lab: plataforma do projeto.
- **Half-Life 2: Raising the Bar**, David Hodgson (2004): como a Valve testa os próprios jogos.
- **A Arte do Game Design** (*The Art of Game Design: A Book of Lenses*), Jesse Schell: importância do playtesting e como fazê-lo.
- **Aseprite**: editor de pixel art usado para desenhar os novelos de lã e o cachorro no disco voador.

Uso de inteligência artificial

Conforme o edital, declaro que usei o Claude (Anthropic) para montar este PDF: diagramação, organização em texto das minhas respostas sobre o projeto e geração das capturas de tela. Também usei o Claude para implementar melhorias feitas no jogo nesta etapa: os retornos visuais descritos em "Polimento" e o modo Build.

Arquivo entregue: `asteroids.sb3`. Capturas de tela feitas do próprio projeto rodando, em 27/09/2026.